

Lab 1: Python Fundamentals and Your First Automation

Points: 75

Overview

This lab has two parts. Part 1 is pure Python (no `arcpy`) working with a plain CSV file. Part 2 is geoprocessing: you'll run the same workflow three ways, the way we did with Buffer in week 2.

Download the template notebook, `lab01_template.ipynb`, before you start. It has one markdown header and one seeded code cell per task below. Complete all of your work inside it, in order, and submit the completed file. Read the whole lab before you start. Remember, later steps build on earlier ones, and it's easier to plan your notebook if you know where it's headed.

Data: `ohio_precip_2025.csv` and `ohio_counties.shp` are in the course GitHub repository. `ohio_precip_2025.csv` has one row per weather station, with columns `station_id`, `name`, `lat`, `lon`, and `precip_mm_2025` (total precipitation recorded at that station in 2025, in millimeters).

Part 1: Python Fundamentals (35 points)

Everything here is plain Python, working directly with the CSV file. In your template notebook, this is the `## Part 1` section — keep your work below its heading.

Reading the CSV

We haven't covered file reading in class yet, so below is an example to get you started. `csv.DictReader` turns each row of a CSV into a dictionary keyed by column name, so you don't have to track column positions by hand:

```
import csv

with open("ohio_precip_2025.csv") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row["station_id"], row["precip_mm_2025"])
        break # just look at the first row for now
```

Important: Save your notebook in the same folder as the CSV file so Python can find it. The plain `open()` function looks for files relative to where your notebook is saved, but it does *not* look in the `arcpy.env.workspace` folder. If you get a `FileNotFoundError` when you run this, use the full path: `open("C:/YOURPATHHERE/ohio_precip_2025.csv")` instead.

Run the above code, then look at what one `row` actually contains, and remove the `break` command once you understand the how the data are structured.

Two things to notice: every value in `row` comes back as a **string**, even `precip_mm_2025`, so you'll need to convert it with `float()` before doing any math with it; and `with open(...) as f:` closes the file for you automatically once the indented block finishes.

Task 1.1: Count, min, max, mean (7 points)

Using the pattern above, you will read every row of `ohio_precip_2025.csv` and compute, for `precip_mm_2025`:

- the number of stations,
- the minimum value,
- the maximum value,
- the mean (average) value.

Do not hardcode any of these numbers. Your code has to compute them from the file's contents, not from a value you looked up, computed in Excel, or typed in — if the data were to change, your code should still give the right answer without you touching it. Print all four values with a label so it's clear what each one is.

Let's work through it in three guided steps:

1. **Before the loop:** we need to create accumulator variables: a running count, a running total, a running minimum, and a running maximum. Think about what each one's starting value should be so the first row updates it correctly.

2. **Inside the loop:** for each row, convert `precip_mm_2025` to `float`, then update every accumulator: add 1 to the count, add the value to the running total, and compare against the running min/max.
3. **After the loop:** compute the mean from your running total and count, and print all four values with clear labels.

Hint: the accumulator pattern

An “accumulator” is just a variable you create before a loop starts and update on every iteration. The running count and running total above are both accumulators. The general shape:

```
count = 0
total = 0.0

for row in reader:
    value = float(row["precip_mm_2025"])
    count = count + 1
    total = total + value

mean = total / count
```

Min and max follow the same pattern, but instead of always adding, you only update when the new value beats the current best-so-far. Ask yourself: what should `running_min` start at, before you’ve seen any real data, so that the very first row is guaranteed to replace it?

Expected output looks like: four printed lines, each with a label and a number — something in the shape of `Station count: <int>`, `Min precip_mm_2025: <float> mm`, `Max precip_mm_2025: <float> mm`, `Mean precip_mm_2025: <float> mm`. (Note, this hint shows you the format, not the values — you’ll compute those from the data yourself.)

Task 1.2: Stations above the mean (7 points)

Using the mean you just computed, find every station whose `precip_mm_2025` is above the mean. Print how many stations that is, and print the name and value for the first five of them.

Guided steps:

1. **Reuse, don’t recompute.** Use the `mean` variable from Task 1.1 — don’t write a second pass that recalculates a mean independently
2. **Filter:** loop over the rows again (or over a list you saved while reading them the first time) and collect the ones whose `precip_mm_2025` is above the mean

3. **Report:** print the count of matches, then print name + value for the first five

Hint: the above-mean filter

You need the raw rows a second time, so either re-open and re-read the CSV, or save each row (or just the fields you need) into a list the first time through, in Task 1.1. Either approach is fine — the design of the filtering step looks like:

```
above_mean = []
for row in rows:
    value = float(row["precip_mm_2025"])
    if value > mean:
        above_mean.append(row)
```

Then `len(above_mean)` is your count, and `above_mean[:5]` gets you the first five to print.

Expected output looks like: one line such as `<int> stations above the mean`, followed by up to five lines, each showing a station name and its value — e.g. `<name>: <float> mm`.

Task 1.3: A conversion function (21 points)

Precipitation totals are more familiar to a lot of students in inches than millimeters. We haven't covered writing your own functions yet (that comes later), but here's the shape of one — copy this pattern:

```
def function_name(parameter):
    return parameter * 2
```

A function definition starts with `def`, a name, and a parenthesized parameter list, followed by a colon. The indented body runs each time the function is called, and `return` hands a value back to whoever called it.

Following that pattern, write a function called `mm_to_inches(mm)` that converts a value in millimeters to inches (1 inch = 25.4 mm). Give it a one-line **docstring** — a quoted string right under the `def` line describing what the function does — and use it inside an f-string to print a one-line report, along the lines of:

```
f"The mean station total was {mean_precip:.1f} mm ({mm_to_inches(mean_precip):.2f} in)."
```

Use `mm_to_inches` at least once more somewhere else in your Part 1 output (for example, converting the min or max value too).

Hint: writing the function definition

Filling in the pattern above with a docstring looks like this shape (the conversion logic is yours to write):

```
def mm_to_inches(mm):  
    """One-line description of what this function does."""  
    return ... # your conversion here
```

Call it the same way you'd call any built-in function, `mm_to_inches(some_value)`, anywhere you'd otherwise write out the conversion math by hand — including inside an f-string's `{}`.

Expected output looks like: at least two printed report lines, each a sentence that names a millimeter value and its inches equivalent in parentheses — e.g. "...was <float> mm (<float> in)." — using two different input values.

Part 2: One Tool, Three Ways — Buffer and Summarize (28 points)

In week 2 you ran Buffer on the state parks layer three ways: the GUI dialog, ModelBuilder, and `arcpy`. This time you're running a small two-step workflow (**Buffer, then a spatial summary**) on `ohio_counties.shp`, and you only need to build it two of the three ways.

The workflow: buffer every county in `ohio_counties.shp` by a distance of your choosing (pick something between 1 and 10 miles and state what you picked), keeping each county's buffer as a separate feature (don't dissolve). Then summarize `ohio_state_parks.shp` within those buffers — you're looking for a tool that counts, for each buffered county, how many state parks fall inside it. Look for a summarizing tool in the Analysis toolbox alongside Buffer; its dialog and its "Learn more" reference page will tell you what its inputs are.

Task 2.1: ModelBuilder (16 points)

Build this two-step workflow — Buffer, then the summary tool, chained so the output of Buffer feeds the input of the summary step — in ModelBuilder. Follow this click path:

1. Open ModelBuilder: **Analysis** tab → **ModelBuilder**, or right-click a toolbox in the Catalog pane → **New** → **Model**.
2. Drag `ohio_counties.shp` from the Catalog pane onto the model canvas as your first input.

3. Search “buffer” in the model’s tool search box, drag **Buffer** onto the canvas, and connect `ohio_counties.shp` into it as the input feature.
4. Double-click the Buffer box to open its parameter dialog *inside the model* — set your chosen distance (1–10 miles) and leave dissolve unset (NONE), so every county keeps its own buffer feature.
5. Search for the summarizing tool you identified above (in the Analysis toolbox, near Buffer), drag it onto the canvas.
6. Connect Buffer’s output into the summarize tool as its input polygons, and set `ohio_state_parks.shp` as the layer being summarized — this is the connection that makes the workflow “chained” rather than two separate runs.
7. Run the model (**Run** button, or **Model** menu → **Run Entire Model**).
8. Add the result to your map if it isn’t added automatically.

Expected result: a model canvas with two tool boxes, both fully colored (not white/unset), connected by an arrow running from Buffer’s output into the summarize tool’s input. You should NOT have disconnected boxes sitting side by side.

Deliverables:

- A screenshot of the finished model (both tool boxes, connected, filled in).
- A screenshot showing the resulting output layer added to your map.

Task 2.2: arcpy (12 points)

Write the same two-step workflow as `arcpy` calls in your notebook. Set your workspace first. **Your solution should take about 10 lines of code or fewer**, not counting blank lines. It’s ok if you have more than that. Print `arcpy.GetMessages()` after the summary step so there’s a record of it running cleanly.

Expected output: a short block of `arcpy` status text (whatever `GetMessages()` returns) with no traceback — if you see a red error block instead, something upstream didn’t run.

Task 2.3: Prose

In a markdown cell, in 3–5 sentences: for this specific two-step workflow, when would you use the GUI dialog, when ModelBuilder, and when `arcpy`? Ground your answer in this task, not just the general tradeoffs from week 2 — is this a workflow you’d run once, hand to someone else, or automate and repeat? (

Deliverables and submission

Submit through Canvas:

1. Your completed template notebook (`lab01_template.ipynb`), containing all of Part 1's code and output, Part 2's `arcpy` code and its printed messages, and your Task 2.3 prose. It should all be in one file, runnable top to bottom.
2. The two ModelBuilder screenshots from Task 2.1.

Submit code as code, not screenshots of code. A screenshot of a code cell can't be run, copied, or checked. Write and run your actual code in the notebook. Screenshots are for the ModelBuilder model and the map view only

Point breakdown

Component	Points
Part 1: Python fundamentals	35
Part 2: Buffer + summarize	28
Prose (Task 2.3)	8
Style and naming	4
Total	75

“Style and naming” covers `snake_case` variable and function names, readable f-strings instead of manual string concatenation, and a docstring on `mm_to_inches` — the same conventions from weeks 1 and 2.